

They are used to find the shortest path between two points in a graph. BFS explores the graph level by level, DFS explores deeply along a branch, and A* uses a heuristic to guide its search intelligently. Here's a brief overview of these algorithms.

Breadth-First Search (BFS)

Concept: BFS explores the graph by examining all neighbours at the current level before moving to the next level.

Implementation: It uses a queue to store nodes that need to be explored.

Pros: Finds the shortest path in unweighted graphs.

Algorithm:

1. Start with the initial node and enqueue it.
2. While the queue is not empty:
 - Dequeue a node.
 - For each unvisited neighbour of the dequeued node:
 - Enqueue the neighbour.
 - Mark the neighbour as visited.

Example: Imagine searching for a friend on a social network. You start with yourself, then check all your friends, then their friends, and so on, level by level.

Depth-First Search (DFS)

Concept: DFS explores as deeply as possible along one path before backtracking.

Implementation: It tracks the explored path using a stack (or recursion).

Pros: Can be useful for finding paths in trees or graphs with cycles.

Algorithm:

1. Start with the initial node and push it onto the stack.
2. While the stack is not empty:
 - Pop a node from the stack.
 - For each unvisited neighbour of the popped node:
 - Push the neighbour onto the stack.
 - Mark the neighbour as visited.

Example: Imagine trying to find the exit of a maze. You

explore one path until you reach a dead end, then backtrack and try another.

A Search*

Concept: A* is an informed search algorithm that uses a heuristic to guide its search and find the optimal path.

Implementation: It uses a heuristic function (e.g., Manhattan distance, Euclidean distance) to estimate the cost of reaching the goal from a given node.

Pros: Efficiently finds the shortest path in many cases, especially when the heuristic is admissible (i.e., never overestimates the actual cost).

Algorithm:

1. Start with the initial node and add it to the open list.
2. While the open list is not empty:
 - Get the node with the lowest estimated cost from the open list.
 - If the current node is the goal node, reconstruct the path and return it.
 - For each neighbour of the current node:
 - Calculate the cost of reaching the neighbour from the initial node.
 - If the neighbour is not in the open or closed list, add it to the open list.
 - If the neighbour is already in the open or closed list, update its cost if the new cost is lower.

Example: Suppose a robot is moving through a space to reach a certain destination. To effectively direct the robot toward the goal, A* may use a heuristic (such as the straight-line distance or Manhattan distance measure).

Lisp programming

A graph, its coordinates, and the costs of moving from one node to another must be defined before we can fairly implement the BFS, DFS, and A* pathfinding algorithms. Let's look at a network with six nodes, as seen in Figure 1, where A and F are the starting and the goal nodes, respectively.

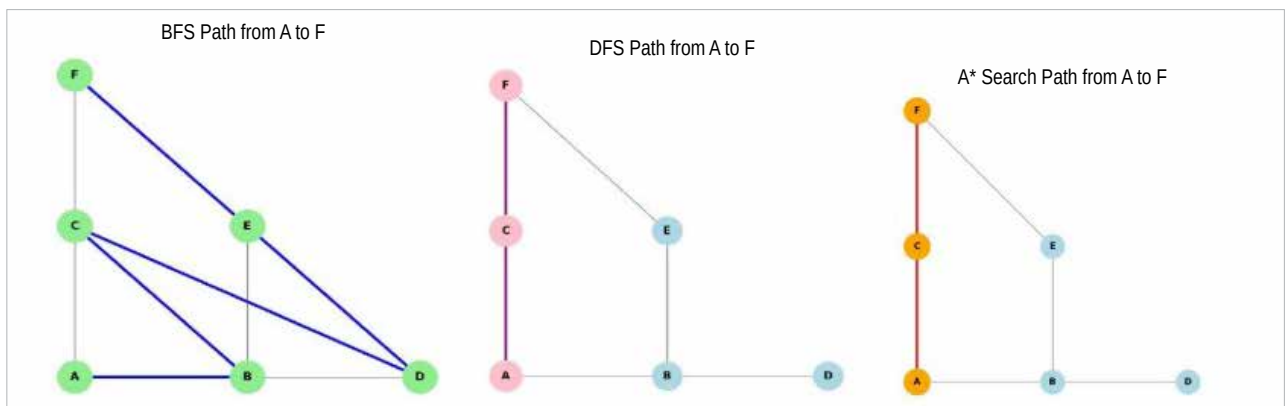


Figure 1: Visual illustration of the given graph ((A, B, C), (B, A, D, E), (C, A, F), (D, B), E, B, F)